

Integrating ESM-2 and Graph Neural Networks with AlphaFold-2 Structures for Enhanced Protein Function Prediction

Thi-Tuyen Nguyen,[#] Zhuocheng Jiang,[#] Van-Nui Nguyen, Nguyen Quoc Khanh Le,^{*} and Matthew Chin Heng Chua^{*}



Cite This: *ACS Omega* 2025, 10, 38103–38111

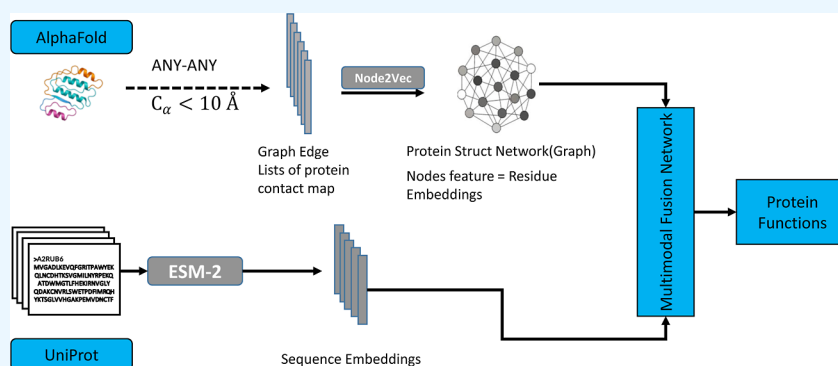


Read Online

ACCESS |

Metrics & More

Article Recommendations



ABSTRACT: Protein function prediction is essential for elucidating biological processes and accelerating drug discovery. However, the vast number of unannotated protein sequences and the limited availability of experimentally validated functional data remain major challenges. Although deep learning models based on protein sequences or protein–protein interaction networks have shown promise, their performance is still restricted, particularly for proteins without interaction data. Furthermore, many existing approaches treat sequence and structural information separately, potentially resulting in suboptimal feature representations. To address these limitations, we propose an improved graph-based framework that integrates two key innovations: (i) ESM-2, a state-of-the-art protein language model, to generate semantically rich sequence embeddings; and (ii) a hybrid pooling mechanism within graph convolutional blocks to better capture both global and local structural features from AlphaFold2-predicted structures. Experiments on the human proteome demonstrate that our model consistently outperforms existing methods in predicting molecular function, cellular component, and biological process annotations. These findings highlight the advantages of combining advanced sequence representations with enhanced structural learning for accurate and generalizable protein function prediction.

1. INTRODUCTION

Protein function prediction is a fundamental task in computational biology, as proteins play central roles in virtually all cellular processes, including catalysis, molecular transport, signal transduction, and gene regulation.^{1,2} Accurate functional annotation of proteins is therefore critical for understanding biological mechanisms and supporting applications such as drug discovery, disease diagnosis, and synthetic biology.³ However, experimental techniques such as enzyme assays, surface plasmon resonance, and gel electrophoresis, while accurate, are labor-intensive, costly, and impractical for large-scale annotation.

The rapid growth of protein databases—exemplified by UniProt, which contains over 100 million sequences but only a small fraction with experimentally verified functions⁴—highlights the urgent need for automated, scalable computational methods. The gene ontology (GO) framework,⁵ which categorizes protein functions into biological processes,

molecular functions, and cellular components, has become the standard for functional annotation.⁶ Traditional sequence alignment tools such as BLAST⁷ and HMMER,⁸ provide valuable insights but often fail to detect functional divergence among structurally distinct yet sequence-similar proteins.

Machine learning, particularly deep learning, has emerged as a powerful alternative. Sequence-based deep learning models leverage architectures such as convolutional neural networks (CNNs), recurrent neural networks (RNNs), and transformers to extract functional signals directly from protein sequences.^{9,10}

Received: June 10, 2025

Revised: August 5, 2025

Accepted: August 11, 2025

Published: August 16, 2025



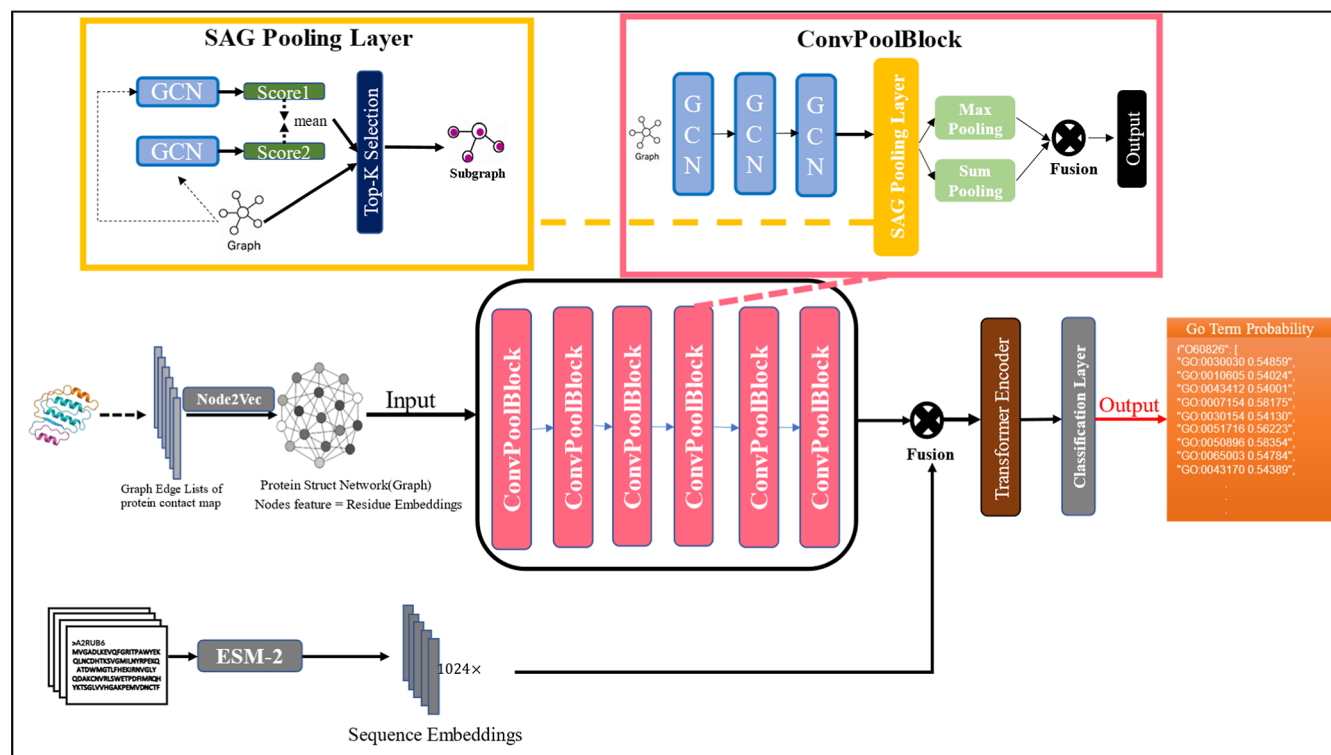


Figure 1. Model structure.

For instance, DeepGO¹¹ uses CNNs and bidirectional long short-term memory (BiLSTM), while models such as ProteinBERT¹² and evolutionary scale modeling (ESM)¹³ employ large transformer-based encoders trained on massive protein sequence data sets. Despite these advances, sequence-only models frequently struggle to distinguish proteins with similar sequences but divergent functions, underscoring the importance of incorporating structural information.

Protein three-dimensional (3D) structures provide critical insights into biochemical activity and molecular interactions. Traditional structure-based methods such as TM-align¹⁴ and DALI¹⁵ align query proteins to annotated templates to infer function, improving accuracy but suffering from high computational costs and reliance on experimentally determined structures. The advent of AlphaFold2¹⁶ has largely overcome this barrier by enabling accurate, large-scale structural predictions, creating new opportunities for structure-aware functional annotation.

Graph-based deep learning methods, particularly graph convolutional networks (GCNs),¹⁷ offer a natural framework for leveraging structural information. By modeling proteins as contact graphs, where residues are nodes and edges represent spatial proximity, GCNs can effectively capture geometric and topological information. These models have been successfully applied to tasks such as protein surface analysis,¹⁸ protein–protein interaction prediction,¹⁹ and model quality assessment.²⁰

For protein function prediction specifically, models such as DeepFRI²¹ and HEAL²² have demonstrated the value of combining structural features with deep learning. HEAL, for example, integrates AlphaFold2-predicted structures¹⁶ and language model embeddings using hierarchical graph transformers and contrastive learning, showcasing the potential of multimodal approaches. However, such models remain isolated

examples; most existing methods still treat sequence and structure separately or rely on shallow feature concatenation, which fails to fully exploit the complementary nature of these modalities. This lack of deep integration limits their ability to learn enriched and discriminative representations.

Moreover, few studies provide clear architectural designs for systematically unifying sequence and structure information. Addressing this gap is the central motivation for our work: to develop an end-to-end multimodal framework that jointly learns from both sequence-level and structure-level inputs to improve predictive accuracy and generalization across diverse protein families.

Building upon the Struct2GO framework,²³ we introduce several key innovations. First, we replace SeqVec with ESM-2 to derive context-rich sequence embeddings. Second, we use Node2Vec²⁴ to encode residue-level structural relationships as informative graph representations. Third, we employ a hybrid pooling strategy combining SAGPool and MixedPooling to capture multiscale structural features. These components are integrated into a unified, computationally efficient architecture.

Our proposed framework (Figure 1) embeds protein sequences using ESM-2 and converts predicted structures into Node2Vec-encoded contact graphs.²⁴ These representations are processed via dual branches—a hybrid-pooled GCN for structure and a CNN for sequence—followed by a transformer encoder for multilabel GO term classification.

This study addresses the limitations of previous approaches²³ by improving both sequence representation and structural feature learning. Our experiments demonstrate that the proposed multimodal framework significantly outperforms benchmark methods, providing a robust and scalable solution for large-scale protein function prediction.

Finally, while multimodal integration introduces additional computational cost, we mitigate this by adopting practical

architectural choices, such as using a moderate-sized ESM-2 variant, precomputing embeddings, and maintaining manageable model depth. These design strategies strike a balance between performance and efficiency, enabling our approach to be applied at proteome scale.

2. MATERIALS AND METHODS

2.1. Data Collection. This study utilizes two primary types of data: protein structural information and protein sequence data. For the development and evaluation of our enhanced model, protein structural data in PDB format were extracted from the AlphaFold2-predicted structures of the human proteome. Corresponding protein sequences were obtained from UniProt,⁴ while functional annotations were sourced from the gene ontology annotation (GOA) project and ontology definitions from the official GO database.

In total, the data set comprises 20,504 proteins annotated with GO terms from the three major ontological categories: molecular function ontology (MFO), biological process ontology (BPO), and cellular component ontology (CCO). GO term mappings were parsed using the go.obo file to maintain consistency with the hierarchical structure of the ontology. To ensure label quality and reduce noise, rare GO terms with low frequency were excluded based on a predefined threshold. After filtering, the resulting data set contained 308 terms for MFO, 310 for CCO, and 713 for BPO.

For benchmarking, we reproduced the original model using the same training, validation, and test splits provided by the original authors. These data sets were preprocessed and partitioned according to their published methodology. The baseline model was retrained using the publicly available source code and hyperparameter configurations reported in the Struct2GO publication.²³

2.2. Protein Representation. Protein representation is achieved by integrating both structural and sequence-based information. Structural data are modeled using contact maps, where amino acids are represented as nodes, and their embeddings are computed using the Node2Vec algorithm.²⁴ For sequence data, embedding vectors are derived from pretrained protein language models.

2.2.1. Graph Construction. Amino acids within a protein are connected by peptide bonds; to model these relationships, protein structure graphs are commonly constructed to facilitate computational analysis and serve as input for deep learning models. In this work, we construct individual graphs based on structural data predicted by AlphaFold-2,¹⁶ with each node representing a single amino acid residue.

To capture spatial relationships between residues, we employ the Any–Any method for graph construction. This approach considers distances between all atomic pairs of two amino acids. Following a widely accepted convention, an edge is established between two nodes if the distance between their respective C α atoms is less than 10 Å. This strategy ensures the preservation of key spatial interactions in the three-dimensional structure, including those between residues that are sequentially distant but spatially proximal due to protein folding.

Representing protein structures as graphs is a critical preprocessing step that facilitates the extraction of spatial features, enhancing the performance of deep learning models in protein function prediction tasks. The resulting contact maps, derived from the constructed edge lists, transform the complex 3D protein structure into a tractable 2D representa-

tion while retaining essential geometric information. Among various methods proposed for contact map construction, the Any–Any method was selected for its superior ability to capture fine-grained spatial relationships within protein structures.

2.2.2. Residue-Level Representation. Accurately representing the spatial features of amino acid residues within a protein requires more than simple one-hot encoding, as it cannot capture the complex three-dimensional relationships between residues.²⁵ To address this limitation, graph-based representation techniques are widely employed, as they enable the extraction of spatially aware embeddings for each node (i.e., residue) in the protein graph.

In this study, we utilize Node2Vec²⁴ to generate residue-level embeddings from the constructed protein graphs. Node2Vec is a network embedding algorithm based on a biased random walk strategy, designed to efficiently explore the graph structure and learn continuous feature representations of nodes. The algorithm performs multiple random walks to sample node sequences, akin to word sequences in natural language processing, and then maximizes the conditional probability between a target node and its surrounding nodes to learn the embedding space.

Node2Vec introduces two hyperparameters, p and q , which govern the behavior of the random walks. Parameter p controls the likelihood of returning to the previous node, influencing the preference for depth-first search (DFS). Meanwhile, q governs the probability of visiting a new node, modulating the exploration between local (BFS-like) and global graph structures. In our implementation, we adopt $p = 0.8$ and $q = 1.2$, consistent with the original Struct2GO setting, to ensure fair comparison and reliable graph embedding quality.

The learning objective of Node2Vec is presented as follows

$$\max_f \sum_{u \in V} \log P_r(N_S(u)|f(u)) \quad (1)$$

In this context, $N_S(u)$ denotes the set of neighboring nodes of node u , determined through a random walk process, and $f(u)$ represents the feature of node u . The original Node2Vec paper employs a negative sampling approximation to optimize the objective function, thereby improving computational efficiency. The probability of transitioning to a specific node during the random walk is defined in the original paper as follows

$$P(c_i = x | c_{i-1} = v) = \begin{cases} \frac{\pi_{vx}}{Z} & \text{if } (v, x) \in E \\ 0 & \text{other wise} \end{cases} \quad (2)$$

In this formula, Z is a normalized constant and π_{vx} is the unnormalized probability of wandering from v to x , which is calculated as follows

$$\pi_{vx} = \alpha_{pq}(t, x) \cdot W_{vx} \quad (3)$$

W_{vx} is the weight of the edge between v and x , $\alpha_{pq}(t, x)$ is adjusted based on the graph shortest path distance d_{tx} from t to x , which is calculated as

$$\alpha_{pq}(t, x) = \begin{cases} \frac{1}{p} & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ \frac{1}{q} & \text{if } d_{tx} = 2 \end{cases} \quad (4)$$

By applying the Node2Vec algorithm to the residue-level protein graph, we obtain informative and spatially contextual embeddings for each amino acid. These embeddings provide a compact and discriminative feature representation of the protein's three-dimensional structure, thereby enhancing the capacity of downstream deep learning models to accurately predict protein function.

2.2.3. Sequence Embeddings. To characterize protein sequence data, we employ both one-hot encoding and embeddings derived from a pretrained protein language model. Recent advances in natural language processing (NLP)-based protein language models have shown strong performance across various protein analysis tasks. Among these, ESM-2,¹³ developed by Meta AI, is one of the most advanced models, built upon the transformer architecture. ESM-2 generates high-resolution embeddings that have been successfully applied to protein structure prediction and functional annotation.

In this study, we focus on generating protein sequence embeddings using the ESM-2 model. ESM-2 is trained on large-scale protein data sets, including UniRef50 and UniRef90, and employs a deep Transformer encoder composed of multiple self-attention layers and feedforward neural networks (FNNs). The core self-attention mechanism is defined as

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (5)$$

where the input embeddings are differentially linearly transformed to obtain Q, K, V , and the dimensions of the keys are d_k . The masked language model (MLM) is then utilized to train by randomly overlaying some of the sequences, with a loss function using a cross entropy, defined as

$$\mathcal{L} = -\frac{1}{M} \sum_{i \in \text{masked}} \log P(S_i | S_{\text{context}}) \quad (6)$$

where the number of masked positions is M , and S_{context} refers to the sequence before and after masking. Assuming the protein sequence is S with a length of L , each amino acid S_i is mapped to its initial embedding e_i . Subsequently, the final sequence embedding h_i is obtained through multilayer Transformer processing, incorporating richer contextual information. Finally, for the entire sequence, the global embedding h_{global} of the corresponding protein is derived by applying average pooling over the embeddings at each position.

In our experiments, we use the *esm2_t33_650M_URS0D* model provided by the official ESM API to extract sequence embeddings. This variant of ESM-2 contains 33 Transformer layers and produces embeddings with a dimensionality of 1024. All computations were performed on an NVIDIA A10 GPU using FP32 precision. We selected the *esm2_t33_650M_URS0D* model as it provides an effective balance between computational efficiency and predictive performance. While larger models such as ESM-2 3B or 15B may yield marginal

improvements in accuracy, they require significantly more memory and training time, which becomes impractical for large-scale experiments. The 650 M variant was thus chosen to ensure feasibility while still capturing rich contextual information from protein sequences.

2.3. Model and Network. Protein function prediction is essentially a multilabel classification problem, in which the model is expected to output multiple GO terms corresponding to a single protein. The model employed in this paper adopts a hierarchical self-attention graph pooling network architecture. Specifically, the model takes as input the protein structure contact map, where the node features are the embeddings of amino acid residues. The convolutional layers then perform feature extraction and pooling. Next, the features extracted from the graph are fused with the global sequence features generated by the ESM-2 model. The fused features are further processed through a transformer encoder. Finally, the output is passed through a fully connected classification layer to predict the protein functional labels.

2.3.1. Convolutional Layer. In this stage, the main task is the learning and pooling of protein structural features. Specifically, the protein contact map with added self-loops and the embeddings of its nodes are input, along with labels input into the constructed label network. The goal is to aggregate the information on similar nodes at the protein structural level and update the features for downstream classification. The graph convolutional layer used in this project is the graph convolutional layer (GraphConv),¹⁷ whose theoretical basis is the approximation of spectral graph convolution, allowing the extension of convolution operations to graph data structures. The forward propagation of the GraphConv layer is as follows

$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}) \quad (7)$$

where $H^{(l)}$ is the node feature matrix of layer l and has size $N \times D_l$ (number of nodes $\times$ layer l feature dimension), $W^{(l)}$ is the weight of layer l , A is the adjacency matrix, $\tilde{A}$ is the adjacency matrix after adding the self-loop, $\tilde{D}$ is the degree matrix of A . Within the activation function is the use of the degree matrix to first normalize the self-loop adjacency matrix, weighted average of the features of each neighbor node, and then multiplied with the feature matrix of the current layer as well as the weight matrix through the activation function to obtain the next layer. This paper utilizes the interface within Deep Graph Library²⁶ to implement the GraphConv layer, which is implemented in the form of a 3-layer GraphConv in the main convolution module.

2.3.2. Self-Attention Graph Pooling Layer. The self-attention mechanism is widely used in deep learning and is highly effective for many problems, especially those involving ordered and complex features, as it enhances the model's ability to capture local features and contextual understanding. In this paper, we adopt self-attention graph pooling (SAGPool)²⁷ combined with a top-k node selection algorithm to construct the self-attention graph pooling layer. SAGPool is a pooling method based on the self-attention mechanism, which reduces the training burden and the size of the graph while retaining key topological and edge features as much as possible in the classification task of complex graph structures. The first step of SAGPool is to compute the attention score z on the input of the graph, which is performed as follows

$$z = \sigma(D^{-1/2} \tilde{A} D^{-1/2} X \theta) \quad (8)$$

$\tilde{A}$ is the neighborhood matrix with added self-loop, D is the degree of this neighborhood matrix, X is the node features, and θ is a one-dimensional weight vector. The nodes of top- k are selected based on the mean value obtained from the self-attention score after two passes through GCN. In the node selection algorithm, k is the pooling ratio, and the selection of nodes is performed based on the average score y of the two head attention scores in the previous step in the following way, the top- k important nodes are selected based on the scores then the new node features and the new with a self-loop adjacency matrix are obtained. In the implementation of this project, k will be chosen 0.75 at last to facilitate the comparison with the original paper

$$idx = \text{top } k(Z, [kN]) \quad (9)$$

$$X_{\text{out}} = X \odot Z_{idx}; A_{\text{out}} = A_{idx} \quad (10)$$

2.3.3. Mix Pooling Layer. In the first two layers of the convolutional module, due to the application of the self-attention graph pooling mechanism and top- k node selection based on graph and node features, the output obtained before the subsequent pooling step is primarily intended to reduce computational load and accelerate the training process, rather than directly affecting the graph features. The pooling method employed in this study is a combination of max pooling and sum pooling. The pooling ratio is set to 0.5, after which the two pooling results are concatenated to form the final output

$$X_{\text{out}} = \frac{1}{N} \sum_{idx=0}^{N-1} X_i || \max(X_i) \quad (11)$$

X_{out} is the i th node feature, and we splice the max pooling result and the sum pooling result in the right side of this formula.

2.4. Evaluation.

$$F_{\max}(\tau) = \max_{\tau \in [0,1]} \left\{ \frac{2^* \text{pr}(\tau)^* \text{rc}(\tau)}{\text{pr}(\tau) + \text{rc}(\tau)} \right\}$$

To comprehensively assess the predictive performance of the proposed model, we employ several widely used evaluation metrics, including AUC, AUPR, and $F_{\max}$. These metrics provide complementary perspectives on the model's ability to accurately predict protein functions. AUC (area under the receiver operating characteristic curve) evaluates the model's ability to distinguish between positive and negative classes by computing the area under the ROC curve, which plots the true positive rate (TPR) against the false positive rate (FPR) across varying thresholds. A higher AUC value indicates better overall classification performance and stronger class separation. AUPR (area under the precision-recall curve): the precision-recall (PR) curve illustrates the trade-off between precision and recall at different decision thresholds. AUPR is particularly informative in settings with class imbalance, as it reflects the model's ability to maintain high precision without sacrificing recall. $F_{\max}$ score represents the maximum F1-score achievable over all thresholds and balances both precision and recall. At a given threshold τ , it computes the mean precision (pr) and mean recall (rc) across the test set. $F_{\max}$ is defined as

$$F_{\max}(\tau) = \max_{\tau \in [0,1]} \left\{ \frac{2^* \text{pr}(\tau)^* \text{rc}(\tau)}{\text{pr}(\tau) + \text{rc}(\tau)} \right\} \quad (12)$$

$$\text{pr}(\tau) = \frac{1}{m(\tau)} \sum_{i=1}^{m(\tau)} \text{pr}_i(\tau) \quad (13)$$

$$\text{rc}(\tau) = \frac{1}{n} \sum_{i=1}^n \text{rc}_i(\tau) \quad (14)$$

where $m(\tau)$ denotes the number of proteins for which the predicted probability of at least one functional label is greater than or equal to threshold τ , and n is the total number of proteins in the test data set. Together, these metrics offer a robust evaluation of the model's performance in multilabel protein function prediction, capturing both classification quality and sensitivity to label imbalance.

2.5. Training and Testing. To comprehensively evaluate the effectiveness of the proposed model, three experiments with distinct training configurations were conducted.

In the first experiment, we fully replicated the methodology, hyperparameters, and model architecture described in the Struct2GO paper.²³ The training, validation, and test data sets for this experiment were obtained directly from the publicly available data set provided by the original authors.

In the second experiment, we introduced several modifications to the original pipeline. First, the ESM-2 model was used to generate protein sequence embeddings, replacing the previously used Seq2Vec embeddings. Additionally, one-hot sequence mappings were excluded from the node feature representation in the protein graph. Furthermore, the pooling strategy in the final pooling layer was altered: instead of the original approach, which applies max pooling twice followed by averaging, we used a combination of sum pooling and max pooling to better capture diverse structural signals. These changes were designed to allow a more direct and meaningful comparison of representational effectiveness.

The third experiment followed the same configuration as the second, with the only difference being the inclusion of one-hot encoded sequence features as part of the node feature input in the protein graph.

For the two experiments involving self-processed data, the data set was randomly partitioned into training, validation, and test sets in a 7:2:1 ratio. The core network architecture was organized into convolutional pooling blocks (ConvPool-Blocks), with each block consisting of a GraphConv layer, a SAGPool layer, and a mixed pooling layer.

After six stacked ConvPoolBlocks, the resulting outputs—consisting of the processed adjacency matrix and node features—were concatenated with the global sequence embeddings. This combined representation was then passed through a Transformer encoder composed of three layers and eight attention heads to further capture high-level interactions. The final output was processed by a three-layer fully connected classification module, which produced probability scores for multiple GO terms per protein.

To evaluate performance, we employed the metrics $F_{\max}$, AUC, and AUPR across all experiments. For the Struct2GO replication experiment, the original preprocessed data set and code provided by the authors were used for training and evaluation, ensuring comparability. Importantly, in the benchmark comparison section, we report the results from our own Struct2GO reproduction experiment rather than the original paper's published results to maintain methodological consistency. For other comparative methods not directly evaluated

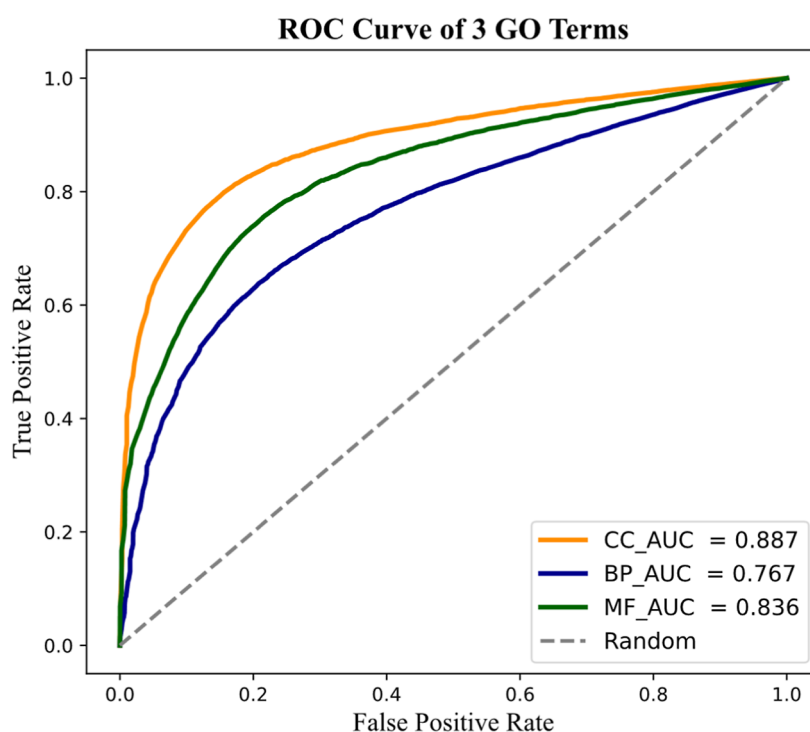


Figure 2. ROC curves of the best models.

Table 1. Model Performance Comparison on MFO, CCO, and BPO^a

model	MFO			CCO			BPO		
	$F_{\max}$	AUC	AUPR	$F_{\max}$	AUC	AUPR	$F_{\max}$	AUC	AUPR
BLAST	0.339	0.577	0.489	0.441	0.563	0.269	0.411	0.623	0.461
DeepGO ¹¹	0.327	0.639	0.571	0.589	0.695	0.448	0.404	0.760	0.625
DEEPPGOA ²⁸	0.385	0.698	0.622	0.629	0.757	0.5	0.477	0.820	0.71
Struct2GO ²³	0.302	0.808	0.256	0.531	0.886	0.601	0.303	0.749	0.261
without one-hot	0.384	0.836	0.418	0.501	0.867	0.483	0.250	0.725	0.194
with one-hot	0.399	0.836	0.421	0.557	0.887	0.600	0.334	0.767	0.305

^aStruct2GO: reproduced result as baseline.

against Struct2GO by its authors, we report the corresponding performance metrics as cited in relevant prior studies.

All experiments were conducted on data sets derived from the human proteome.

3. RESULTS & DISCUSSIONS

3.1. Performance Overview. Figure 2 presents ROC curves for the best-performing models on each GO category. The highest AUC values were achieved in the CCO (0.887), followed by MFO (0.836) and BPO (0.767), reflecting improved discrimination ability across all ontologies.

Hyperparameter tuning revealed that a dropout rate of 0.2 and batch size of 32 yielded optimal performance for MFO and CCO, while a dropout of 0.3 and batch size of 64 were most effective for BPO. Notably, models incorporating one-hot residue features consistently outperformed those without them, reinforcing the benefit of combining high-resolution sequence embeddings with positional encoding in the graph input.

3.2. Comparative Analysis with Existing Methods. Table 1 summarizes the model performance across all GO categories and compares it against widely used baselines including BLAST, DeepGO,¹¹ DEEPPGOA,²⁸ and the reproduced Struct2GO baseline.²³ We have also performed McNemar tests to see the significant differences between our

model and others. The results showed that we achieved better performance than others in most of measurement metrics.

3.3. Effect of Proposed Enhancements. Our results demonstrate consistent performance gains across all GO categories when applying the two key enhancements.

- ESM-2 embeddings significantly outperform SeqVec, likely due to ESM-2's deeper architecture and larger training corpus, which enable more effective capture of evolutionary and functional motifs. Prior work by Antegnini et al.²⁹ showed that ESM-1b outperforms SeqVec in generating protein embeddings. As ESM-2 is a more advanced successor to ESM-1b, it was adopted in this study as a replacement for SeqVec with the expectation of improved performance—a hypothesis that was validated by our experimental results. These findings underscore the critical role of high-quality sequence feature extraction in advancing protein function prediction.
- Hybrid pooling (sum + max) enables richer aggregation of structural information compared to the original pooling design, contributing to performance stability.³⁰
- One-hot residue encoding, when added to the graph node features, consistently boosts performance,³¹

particularly in MFO and BPO tasks where positional context appears more functionally informative.

In particular, for MFO, the $F_{\max}$ improved from 0.302 (Struct2GO) to 0.399, AUC increased from 0.808 to 0.836, and AUPR rose from 0.256 to 0.421. Similar trends were observed in CCO and BPO, highlighting the robustness of the enhancements. Even without the one-hot features, our ESM-2-based model outperforms Struct2GO in most metrics, underscoring the strength of upgraded sequence embeddings alone.

These results reinforce the hypothesis that multimodal learning, when designed carefully, can bridge gaps left by unimodal sequence or structure-based approaches. Our findings align with previous studies advocating for representation learning from both spatial and sequential protein features, but further push the boundary through improved architectural design.

3.4. Limitations and Future Work. While the proposed framework demonstrates notable performance gains, it inevitably incurs increased computational cost due to the integration of multiple modalities and model components. To mitigate this overhead, several strategies were adopted. First, we selected a moderate-sized variant of ESM-2, balancing representation quality with inference speed. Second, both ESM-2 sequence embeddings and Node2Vec-based structural embeddings were precomputed, reducing runtime during model training. Third, we constrained the depth of the graph convolutional and Transformer layers to prevent unnecessary model complexity. Collectively, these design choices reduced memory usage and training time, improving scalability and practicality for large-scale or cross-species protein function prediction.

Future work may focus on applying model compression techniques or developing lightweight architectures to further enhance computational efficiency without compromising predictive accuracy.

Despite the overall performance improvements across GO categories, the model exhibited relatively lower performance in BPO category, with reduced $F_{\max}$, AUC, and AUPR scores compared to the MFO and CCO categories. This limitation may stem from the inherent complexity and semantic heterogeneity of biological processes, which often involve multistep interactions and diverse functional contexts. Additionally, the label distribution in BPO is more imbalanced, with many rare terms underrepresented in the training data, making it more difficult for the model to learn informative patterns. Addressing this limitation may require class-aware training objectives or the integration of biological pathway knowledge to improve functional discrimination and representation within the BPO category.

4. CONCLUSION

We developed an enhanced graph-based framework for protein function prediction by integrating ESM-2 sequence embeddings, hybrid structural pooling, and one-hot residue encoding. Compared to the reproduced Struct2GO baseline, our model consistently improved performance across MFO, CCO, and BPO prediction, highlighting the benefit of combining advanced protein language models with refined graph-based strategies. These improvements are biologically meaningful, supporting more accurate functional annotation and potential drug target discovery. Although trained on human proteins, the

framework's design—featuring ESM-2 embeddings and residue-level representations—is inherently generalizable to other species, owing to the strong cross-species transferability of ESM-2. Future directions include expanding training to cross-species data sets, integrating PTM and protein–protein interaction data, exploring SE-equivariant GNNs, and optimizing computational efficiency through lighter ESM variants and graph sampling strategies. This work establishes a robust foundation for scalable, multimodal, structure-aware protein function prediction across diverse biological contexts.

■ ASSOCIATED CONTENT

Data Availability Statement

All data sets, models, and source code used in this study are publicly available at: <https://github.com/khanhlee/esm-alpha>.

■ AUTHOR INFORMATION

Corresponding Authors

Nguyen Quoc Khanh Le – *In-Service Master Program in Artificial Intelligence in Medicine, College of Medicine, Taipei Medical University, Taipei 110, Taiwan; AIBioMed Research Group, Taipei Medical University, Taipei 110, Taiwan; Translational Imaging Research Center, Taipei Medical University Hospital, Taipei 110, Taiwan; orcid.org/0000-0003-4896-7926; Email: khanhlee@tmu.edu.tw*

Matthew Chin Heng Chua – *Department of Biomedical Informatics, Yong Loo Lin School of Medicine, National University of Singapore, Singapore 119228, Singapore; Email: matthewchua@nus.edu.sg*

Authors

Thi-Tuyen Nguyen – *University of Information and Communication Technology, Thai Nguyen University, Thai Nguyen 25000, Viet Nam*

Zhuocheng Jiang – *Department of Biomedical Informatics, Yong Loo Lin School of Medicine, National University of Singapore, Singapore 119228, Singapore*

Van-Nui Nguyen – *University of Information and Communication Technology, Thai Nguyen University, Thai Nguyen 25000, Viet Nam*

Complete contact information is available at: <https://pubs.acs.org/10.1021/acsomega.5c05484>

Author Contributions

#T.-T.N. and Z.J. equal contributions. T.-T.N.: conceptualization, validation, formal analysis, investigation, writing—original draft preparation, visualization. Z.J.: conceptualization, methodology, validation, formal analysis, investigation, writing—original draft preparation, visualization. V.N.N.: validation, investigation, data curation. N.Q.K.L.: conceptualization, methodology, validation, writing—review and editing, visualization, supervision, funding acquisition. M.C.H.C.: methodology, validation, writing—review and editing, supervision, funding acquisition. All authors have read and agreed to the published version of the manuscript.

Funding

This work is supported by the National Science and Technology Council, Taiwan [grant number NSTC114–2221-E-038-015].

Notes

The authors declare no competing financial interest.

REFERENCES

- (1) Alberts, B.; Bray, D.; Hopkin, K.; Johnson, A. D.; Lewis, J.; Raff, M.; Roberts, K.; Walter, P. *Essential cell biology*. Garland Science; Oxford University Press, 2015.
- (2) Nelson, D. L.; Lehninger, A. L.; Cox, M. M. *Lehninger Principles of Biochemistry*; Macmillan, 2008.
- (3) Carnielli, C. M.; Winck, F. V.; Paes Leme, A. F. Functional annotation and biological interpretation of proteomics data. *Biochim. Biophys. Acta* **2015**, 1854 (1), 46–54.
- (4) Bateman, A.; Martin, M. J.; Orchard, S.; Magrane, M.; Ahmad, S.; Alpi, E.; Bowler-Barnett, E. H.; Britto, R.; Bye-A-Jee, H.; Cukura, A.; et al. UniProt: the Universal Protein Knowledgebase in 2023. *Nucleic Acids Res.* **2023**, 51 (D1), D523–D531.
- (5) Ashburner, M.; Ball, C. A.; Blake, J. A.; Botstein, D.; Butler, H.; Cherry, J. M.; Davis, A. P.; Dolinski, K.; Dwight, S. S.; Eppig, J. T.; Harris, M. A.; Hill, D. P.; Issel-Tarver, L.; Kasarskis, A.; Lewis, S.; Matese, J. C.; Richardson, J. E.; Ringwald, M.; Rubin, G. M.; Sherlock, G. Gene ontology: tool for the unification of biology. The Gene Ontology Consortium. *Nat. Genet.* **2000**, 25 (1), 25–29.
- (6) Aleksander, S. A.; Balhoff, J.; Carbon, S.; Cherry, J. M.; Drabkin, H. J.; Ebert, D.; Feuermann, M.; Gaudet, P.; Harris, N. L.; Hill, D. P.; Lee, R.; Mi, H.; Moxon, S.; Mungall, C. J.; Muruganugan, A.; Mushayahama, T.; Sternberg, P. W.; Thomas, P. D.; Van Auken, K.; Ramsey, J.; Siegle, D. A.; Chisholm, R. L.; Fey, P.; Aspromonte, M. C.; Nugnes, M. V.; Quaglia, F.; Tosatto, S.; Giglio, M.; Nadendla, S.; Antonazzo, G.; Attrill, H.; dos Santos, G.; Marygold, S.; Strelets, V.; Tabone, C. J.; Thurmond, J.; Zhou, P.; Ahmed, S. H.; Asanithong, P.; Luna Buitrago, D.; Erdol, M. N.; Gage, M. C.; Ali Kadhum, M.; Li, K. Y. C.; Long, M.; Michalak, A.; Pesala, A.; Pritazahra, A.; Saverimuttu, S. C. C.; Su, R.; Thurlow, K. E.; Lovering, R. C.; Logie, C.; Oliferenko, S.; Blake, J.; Christie, K.; Corbani, L.; Dolan, M. E.; Drabkin, H. J.; Hill, D. P.; Ni, L.; Sitnikov, D.; Smith, C.; Cuzick, A.; Seager, J.; Cooper, L.; Elser, J.; Jaiswal, P.; Gupta, P.; Jaiswal, P.; Naithani, S.; Lera-Ramirez, M.; Rutherford, K.; Wood, V.; De Pons, J. L.; Dwinell, M. R.; Hayman, G. T.; Kaldunski, M. L.; Kwitek, A. E.; Laulederkind, S. J. F.; Tutaj, M. A.; VEDI, M.; Wang, S. J.; D'Eustachio, P.; Aimo, L.; Axelsen, K.; Bridge, A.; Hyka-Nouspikel, N.; Morgat, A.; Aleksander, S. A.; Cherry, J. M.; Engel, S. R.; Karra, K.; Miyasato, S. R.; Nash, R. S.; Skrzypek, M. S.; Weng, S.; Wong, E. D.; Bakker, E.; Berardini, T. Z.; Reiser, L.; Auchincloss, A.; Axelsen, K.; Argoud-Puy, G.; Blatter, M. C.; Boutet, E.; Breuza, L.; Bridge, A.; Casals-Casas, C.; Coudert, E.; Estreicher, A.; Livia Famiglietti, M.; Feuermann, M.; Gos, A.; Gruaz-Gumowski, N.; Hulo, C.; Hyka-Nouspikel, N.; Jungo, F.; Le Mercier, P.; Lieberherr, D.; Masson, P.; Morgat, A.; Pedruzzi, I.; Pourcel, L.; Poux, S.; Rivoire, C.; Sundaram, S.; Bateman, A.; Bowler-Barnett, E.; Bye-A-Jee, H.; Denny, P.; Ignatchenko, A.; Ishtiaq, R.; Lock, A.; Lussi, Y.; Magrane, M.; Martin, M. J.; Orchard, S.; Raposo, P.; Speretta, E.; Tyagi, N.; Warner, K.; Zaru, R.; Diehl, A. D.; Lee, R.; Chan, J.; Diamantakis, S.; Raciti, D.; Zarowiecki, M.; Fisher, M.; James-Zorn, C.; Ponferrada, V.; Zorn, A.; Ramachandran, S.; Ruzicka, L.; Westerfield, M.; Aleksander, S. A.; et al. The Gene Ontology knowledgebase in 2023. *Genetics* **2023**, 224 (1), iyad031.
- (7) Altschul, S. F.; Gish, W.; Miller, W.; Myers, E. W.; Lipman, D. J. Basic local alignment search tool. *J. Mol. Biol.* **1990**, 215 (3), 403–410.
- (8) Finn, R. D.; Clements, J.; Eddy, S. R. HMMER web server: interactive sequence similarity searching. *Nucleic Acids Res.* **2011**, 39 (suppl), W29–W37.
- (9) Le, N. Q. K.; Yapp, E. K. Y.; Nagasundaram, N.; Chua, M. C. H.; Yeh, H.-Y. Computational identification of vesicular transport proteins from sequences using deep gated recurrent units architecture. *Comput. Struct. Biotechnol. J.* **2019**, 17, 1245–1254.
- (10) Zhao, Z.; Gui, J.; Yao, A.; Le, N. Q. K.; Chua, M. C. H. Improved Prediction Model of Protein and Peptide Toxicity by Integrating Channel Attention into a Convolutional Neural Network and Gated Recurrent Units. *ACS Omega* **2022**, 7 (44), 40569–40577.
- (11) Kulmanov, M.; Khan, M. A.; Hoehndorf, R.; Wren, J. DeepGO: predicting protein functions from sequence and interactions using a deep ontology-aware classifier. *Bioinformatics* **2018**, 34 (4), 660–668.
- (12) Brandes, N.; Ofer, D.; Peleg, Y.; Rappoport, N.; Linial, M. ProteinBERT: a universal deep-learning model of protein sequence and function. *Bioinformatics* **2022**, 38 (8), 2102–2110.
- (13) Lin, Z.; Akin, H.; Rao, R.; Hie, B.; Zhu, Z.; Lu, W.; Smetanin, N.; Verkuil, R.; Kabeli, O.; Shmueli, Y.; Dos Santos Costa, A.; Fazel-Zarandi, M.; Sercu, T.; Candido, S.; Rives, A. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* **2023**, 379 (6637), 1123–1130.
- (14) Zhang, Y.; Skolnick, J. TM-align: a protein structure alignment algorithm based on the TM-score. *Nucleic Acids Res.* **2005**, 33 (7), 2302–2309.
- (15) Holm, L.; Laiho, A.; Toronen, P.; Salgado, M. DALI shines a light on remote homologs: One hundred discoveries. *Protein Sci.* **2023**, 32 (1), No. e4519.
- (16) Jumper, J.; Evans, R.; Pritzel, A.; Green, T.; Figurnov, M.; Ronneberger, O.; Tunyasuvunakool, K.; Bates, R.; Zidek, A.; Potapenko, A.; Bridgland, A.; Meyer, C.; Kohl, S. A. A.; Ballard, A. J.; Cowie, A.; Romera-Paredes, B.; Nikolov, S.; Jain, R.; Adler, J.; Back, T.; Petersen, S.; Reiman, D.; Clancy, E.; Ziliński, M.; Steinegger, M.; Pacholska, M.; Berghammer, T.; Bodenstein, S.; Silver, D.; Vinyals, O.; Senior, A. W.; Kavukcuoglu, K.; Kohli, P.; Hassabis, D. Highly accurate protein structure prediction with AlphaFold. *Nature* **2021**, 596 (7873), 583–589.
- (17) Kipf, T. N.; Welling, M. In Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*, 2017, p 2017.
- (18) Fout, A.; Byrd, J.; Shariat, B.; Ben-Hur, A. Protein interface prediction using graph convolutional networks. In *Advances in neural information processing systems*, 2017, 30.
- (19) Jha, K.; Saha, S.; Singh, H. Prediction of protein–protein interaction using graph neural networks. *Sci. Rep.* **2022**, 12 (1), 8360.
- (20) Baldassarre, F.; Menendez Hurtado, D.; Eloffsson, A.; Azizpour, H. GraphQA: protein model quality assessment using graph convolutional networks. *Bioinformatics* **2021**, 37 (3), 360–366.
- (21) Gligorijević, V.; Renfrew, P. D.; Kosciółek, T.; Leman, J. K.; Berenberg, D.; Vatanen, T.; Chandler, C.; Taylor, B. C.; Fisk, I. M.; Vlamakis, H.; et al. Structure-based protein function prediction using graph convolutional networks. *Nat. Commun.* **2021**, 12 (1), 3168.
- (22) Gu, Z.; Luo, X.; Chen, J.; Deng, M.; Lai, L. Hierarchical graph transformer with contrastive learning for protein function prediction. *Bioinformatics* **2023**, 39 (7), btad410.
- (23) Jiao, P.; Wang, B.; Wang, X.; Liu, B.; Wang, Y.; Li, J. Struct2GO: protein function prediction based on graph pooling algorithm and AlphaFold2 structure information. *Bioinformatics* **2023**, 39 (10), btad637.
- (24) Grover, A.; Leskovec, J. node2vec: Scalable feature learning for networks. *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining* **2016**, 2016, 855–864.
- (25) Sua, J. N.; Lim, S. Y.; Yulius, M. H.; Su, X.; Yapp, E. K. Y.; Le, N. Q. K.; Yeh, H.-Y.; Chua, M. C. H. Incorporating convolutional neural networks and sequence graph transform for identifying multilabel protein Lysine PTM sites. *Chemometr. Intell. Lab. Syst.* **2020**, 206, 104171.
- (26) Zheng, D.; Wang, M.; Gan, Q.; Song, X.; Zhang, Z.; Karypis, G. Scalable Graph Neural Networks with Deep Graph Library. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining, Association for Computing Machinery*; Virtual Event: Israel, 2021, pp 1141–1142.
- (27) Lee, J.; Lee, I.; Kang, J. In *Self-Attention Graph Pooling, International Conference on Machine Learning*, 2019; pmlr: 2019; pp 3734–3743.
- (28) Zhou, G.; Wang, J.; Zhang, X.; Yu, G. In DeepGOA: predicting gene ontology annotations of proteins via graph convolutional network, *2019 IEEE international conference on bioinformatics and biomedicine (BIBM)*; IEEE: 2019; pp 1836–1841.

(29) Anteghini, M.; Santos, V. A. M. d.; Saccenti, E. PortPred: exploiting deep learning embeddings of amino acid sequences for the identification of transporter proteins and their substrates. *J. Cell. Biochem.* **2023**, *124* (11), 1803–1824.

(30) Dong, Z.; Hu, Q.; Zhang, Z.; Guo, Y.; Cordy, M.; Papadakis, M.; Traon, Y. L.; Zhao, J. On the effectiveness of hybrid pooling in mixup-based graph learning for language processing. *J. Syst. Softw.* **2024**, *216*, 112139.

(31) Tng, S. S.; Le, N. Q. K.; Yeh, H.-Y.; Chua, M. C. H. Improved Prediction Model of Protein Lysine Crotonylation Sites Using Bidirectional Recurrent Neural Networks. *J. Proteome Res.* **2022**, *21* (1), 265–273.



CAS INSIGHTS™

EXPLORE THE INNOVATIONS SHAPING TOMORROW

Discover the latest scientific research and trends with CAS Insights. Subscribe for email updates on new articles, reports, and webinars at the intersection of science and innovation.

Subscribe today

CAS
A division of the American Chemical Society